SESSION 13

TASK 1:

```
use mybd;
Select * from Orders;
CREATE TABLE Orders
   (order_id INT PRIMARY KEY,
   user_id INT NOT NULL,
   order_amount DECIMAL(10, 2) NOT NULL,
   app_name VARCHAR(50) NOT NULL
);

-- 2. Insert sample dataset (10 records)
INSERT INTO Orders (order_id, user_id, order_amount, app_name) VALUES
(101, 1, 450.00, 'Zomato'),
(102, 2, 1200.50, 'Flipkart'),
(103, 1, 280.00, 'Swiggy'),
(104, 3, 3500.00, 'Flipkart'),
(105, 4, 150.00, 'Zomato'),
(106, 2, 620.00, 'Swiggy'),
(107, 5, 890.00, 'Zomato'),
(108, 3, 210.00, 'Swiggy'),
(109, 6, 4500.00, 'Flipkart'),
(110, 4, 310.00, 'Zomato');

Select * from Orders;
 use mydb;
```

Query Output Result

order_id	user_id	app_name	order_amount	total_order_amount
101	1	Zomato	450.00	12110.50
102	2	Flipkart	1200.50	12110.50
103	1	Swiggy	280.00	12110.50
104	3	Flipkart	3500.00	12110.50
105	4	Zomato	150.00	12110.50
106	2	Swiggy	620.00	12110.50
107	5	Zomato	890.00	12110.50
108	3	Swiggy	210.00	12110.50
109	6	Flipkart	4500.00	12110.50
110	4	Zomato	310.00	12110.50

TASK 2:

```
use mydb;
select * from Ord;
CREATE TABLE Ord (
   order_id INT PRIMARY KEY,
   user_id INT NOT NULL,
   order_amount DECIMAL(10, 2) NOT NULL
);

-- Insert sample data
INSERT INTO Ord (order_id, user_id, order_amount) VALUES
(101, 1, 100.00),
(102, 1, 200.00),
(103, 2, 50.00),
```

(104, 2, 150.00),

(105, 2, 100.00),

(106, 3, 300.00);

```
Select
order_id,
   user_id,
   order_amount,
   AVG(order_amount) OVER (PARTITION BY user_id) AS avg_user_order_amount
FROM
Ord;
```

SESSION 13 -TASK 2 FINAL OUTPUT.csv

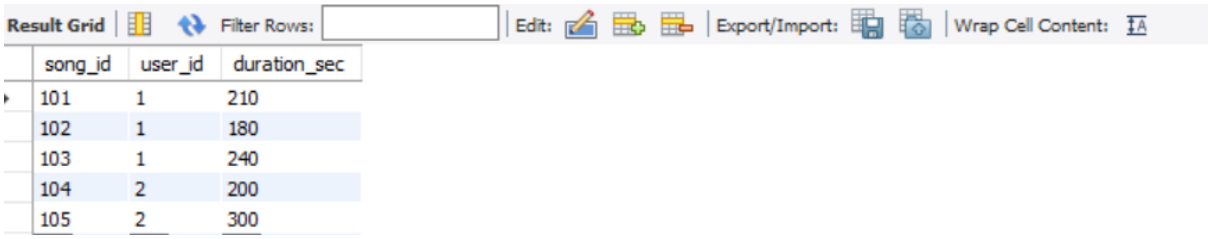
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	user_id	order_amount	avg_user_order_amount
101	1	100.00	150.000000
102	1	200.00	150.000000
103	2	50.00	100.000000
104	2	150.00	100.000000
105	2	100.00	100.000000
106	3	300.00	300.000000

Result 31

TASK 3:

SESSION 13 -TASK 3
musiclists(playlists) ta



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

song_id	user_id	duration_sec
101	1	210
102	1	180
103	1	240
104	2	200
105	2	300

```
use mydb;
select * from Musiclists;
CREATE TABLE Musiclists (
   song_id INT PRIMARY KEY,
   user_id INT NOT NULL,
   duration_sec INT NOT NULL
);
-- Insert sample records
INSERT INTO Musiclists (song_id, user_id, duration_sec)
VALUES
   (101, 1, 210),
   (102, 1, 180),
   (103, 1, 240),
   (104, 2, 200),
   (105, 2, 300);
SELECT
   song_id,
   user_id,
   duration_sec,
   SUM(duration_sec) OVER (PARTITION BY user_id) AS total_user_duration_sec
```

FROM

SESSION 13 -TASK 3
output created.csv

musiclists;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

song_id	user_id	duration_sec	total_user_duration_sec
101	1	210	630
102	1	180	630
103	1	240	630
104	2	200	500
105	2	300	500

TASK 4:

SESSION 13 -TASK 4
,Tabe created. csv.csv

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

rating_id	user_id	movie_name	rating
1	101	Inception	5
2	102	Inception	3
3	103	Inception	4
4	101	Interstellar	4
5	104	Interstellar	2

use mydb;

Select * from Movieratings;

CREATE TABLE MovieRatings (

rating_id INT PRIMARY KEY,

user_id INT NOT NULL,

movie_name VARCHAR(255) NOT NULL,

rating INT CHECK (rating BETWEEN 1 AND 5)

```
);

-- Insert sample data
INSERT INTO MovieRatings (rating_id, user_id, movie_name, rating) VALUES
(1, 101, 'Inception', 5),
(2, 102, 'Inception', 3),
(3, 103, 'Inception', 4),
(4, 101, 'Interstellar', 4),
(5, 104, 'Interstellar', 2),
(6, 102, 'The Dark Knight', 5),
(7, 103, 'The Dark Knight', 5);
Select * from Movieratings;

Select
rating_id,user_id, movie_name,rating,
Avg(rating)Over(partition by movie_name) as Avg_movie_rating,
rating - AVG(rating) OVER(PARTITION BY movie_name) AS rating_difference
FROM
  MovieRatings;
```



SESSION 13 -TASK 4..csv

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

rating_id	user_id	movie_name	rating	Avg_movie_rating	rating_difference
1	101	Inception	5	4.0000	1.0000
2	102	Inception	3	4.0000	-1.0000
3	103	Inception	4	4.0000	0.0000
4	101	Interstellar	4	3.0000	1.0000
5	104	Interstellar	2	3.0000	-1.0000
6	102	The Dark Knight	5	5.0000	0.0000